

TP 01 — Grammar & Lexical Analysis

Team: Anshul · *Partner name* — **Course:** Formal Approaches, Languages and Compilers (ISEP)

1. Language overview

Our language is a C++-like pseudocode with no functions or classes. A program is a sequence of statements. It supports every **minimum feature** on slide 21: `int` declarations, assignment, `if`, `while`, the comparisons `>` `<` `>=` `<=` `==` `!=`, the arithmetic operators `+` `-` `*` `/`, and output with `print(expr)` and `prints("text")`. It also supports all the **recommended extensions** on slide 22: declaration with initialisation, `//` and `/* */` comments, `++` `--` `+=` `-=`, for loops and the logical operators `&&` `||` `!`. We also added `else / else if`, unary minus and multiple declarations (`int a, b = 2;`).

2. Grammar design (Task 01, `grammar/grammar.txt`)

Why EBNF. EBNF's `{ }` (repetition) and `[ ]` (optional) remove the helper non-terminals that CNF needs. A list of statements is simply `{ Statement }`, which makes the grammar short and readable.

Program structure. `Program = { Statement }`, so there is no `main`, as in the slide 20 example. A `Block ( { ... } )` is also a statement, and every `if`, `while` and `for` body must be a block. This removes the "dangling else" ambiguity: an `else` always follows a `}`, so it is clear which `if` it belongs to.

Assignment without `;`. `Assignment ( x = e, x += e, x++, ++x )` does not include the semicolon. `AssignStatement = Assignment ";"` adds it. This lets us reuse the same rule in the `for` header (`for ( i = 0; i < n; i++ )`), where the update part has no `;`.

`=` vs `==`. Slide 21 lists `=` among the logical operators. We chose to keep `=` for **assignment only** and use `==` for comparison, as in C++. Otherwise `if ( a = b )` would be ambiguous.

`print` vs `prints`. We kept two keywords, matching the slide examples. `print` takes an integer expression, and `prints` takes a string literal. Because they are separate keywords, the later stages know the argument type from the syntax alone (integer printing vs string printing in NASM).

Operator precedence and associativity. Precedence comes from the structure of the grammar. Each level only uses the next, stronger level:

Level (weak → strong)	Rule	Operators
1	Condition	<code> </code>
2	AndCondition	<code>&&</code>
3	NotCondition	<code>!</code>
4	Comparison	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>
5	Expression	<code>+</code> <code>-</code>
6	Term	<code>*</code> <code>/</code>
7	Factor	number, variable, <code>(...)</code> , unary <code>-</code>

For `a + 4 * b`, the `*` must be grouped inside a `Term` before `Expression` can combine it with `+`. This gives the same tree as slide 14. The repetition `Term { AddOp Term }` reads left to right, so the operators are **left-associative**: `a - b - c = (a - b) - c`. The comparison is optional (`Expression [CompareOp Expression]`), so `while (n)` is allowed (true if non-zero), as in C.

3. Scanner design (Task 02, `scanner/scanner.l`)

The scanner is written in FLEX in **C++ mode** (`%option c++`). It is built with `flex --c++ scanner.l` and then `g++ -o scanner lex.yy.cc -lfl`. It prints one token per line as `TOKEN_NAME: lexeme`. On the slide 39 example, the output is identical to the slide.

Tokens	Pattern
INT/IF/ELSE/WHILE/FOR/PRINT/PRINTS_TOKEN	the keyword itself
VAR_TOKEN	<code>[A-Za-z] ([A-Za-z] [0-9] _)*</code>
NUMBER_TOKEN	<code>[0-9]+</code>
STRING_TOKEN	<code>"(\\. [^\\"\\n])*" (allows \" inside)</code>
COMPARISON_TOKEN	<code>==</code> <code>!=</code> <code>>=</code> <code><=</code> <code>></code> <code><</code>
ASSIGN, PLUS, MINUS, MULT, DIV, NOT_TOKEN	<code>=</code> <code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>!</code>

Tokens	Pattern
INCREMENT, DECREMENT, PLUS_ASSIGN, MINUS_ASSIGN, AND, OR_TOKEN	++ -- += -= &&
L1/R1/L2/R2/SEMICOLON/COMMA_TOKEN	() { } ; ,

How FLEX chooses between rules. (1) *Longest match*: >=, ==, ++ and += each become one token and are not split into two. (2) *First rule wins on a tie*: the keywords are written before {ID}, so while is WHILE_TOKEN, while whilex (a longer match) is still VAR_TOKEN. prints is placed before print for readability, but longest match would handle it anyway.

Comments and white space are removed in the scanner, so the parser never sees them. Block comments use an **exclusive start condition** %x COMMENT: after /*, only the comment rules are active until */. This handles comments that span several lines.

Error handling. A final catch-all rule . reports any unknown character together with its line number (%option yylineno). Scanning then continues, so all errors are listed at once. Specific messages are given for an identifier starting with a digit (9lives), an unterminated string, and an unterminated block comment. The program exits with code 1 if any error was found.

4. Testing

make check runs the scanner on every file in tests/ and compares the output with tests/expected/. The files are: the slide 20 program, the slide 39 program (output identical to the slide), all operators, all extensions, and an error file. For the error file, the check expects the errors to be detected.